

OPEN SOURCE FRAMEWORK SPECIFICATION

TICKETRIS

An Open-Source, Zero-Training Agentic Ticket Enrichment & Routing Framework Powered by the Model Context Protocol (MCP)

Document Architecture Version: v1.0.0

Date of Publication: May 23, 2026

Licensing Agreement: GNU General Public License v3.0 (GPL-3.0)

Target Architecture: Technology-Neutral / Model Context Protocol Compliant

Classification: Public Technical Whitepaper & Architecture Standard

1. Executive Summary

Modern Enterprise IT Service Management (ITSM) systems are bottlenecked by human error at the point of ingestion. End users, seeking immediate resolution to technical blockers, routinely provide unstructured, ambiguous, or vastly incomplete descriptions when logging incidents or service requests via tools such as ServiceNow, Jira Service Management, or custom help desks. This structural information scarcity triggers a cascade of operational inefficiencies, specifically localized as "ticket ping-pong"—the continuous, manual re-routing of incidents between specialized fulfillment groups. This back-and-forth dramatically decreases the productivity of task resolvers, inflates Mean Time to Resolution (MTTR), and compromises organizational efficiency.

Ticketris is an open technical framework designed to eliminate this structural friction. It acts as a real-time, non-invasive, agentic orchestration layer sitting between the end user and the core enterprise ticketing system. Instead of following historical patterns that rely on training or fine-tuning monolithic Large Language Models (LLMs) on sensitive, volatile corporate data registries, Ticketris leverages an entirely runtime-driven, zero-training architectural philosophy.

By utilizing the open-standard **Model Context Protocol (MCP)**, Ticketris enables an off-the-shelf LLM to act as an active runtime client. When an end user supplies a brief, unstructured issue description, the Ticketris engine coordinates across decentralized internal data repositories through independent, locally deployed MCP servers. It dynamically polls Configuration Management Databases (CMDBs), project registry schemas, developer ownership matrices, repository manifests, and internal knowledge bases to contextually synthesize, validate, and enrich the incident profile. Crucially, the system features a rigid *Human-in-the-Loop (HITL) Validation State*, suspending direct ITSM database mutation until the end user explicitly reviews, amends, and approves the structured, auto-filled payload. Distributed under the GNU General Public License v3.0, Ticketris provides a secure, vendor-agnostic, and privacy-preserving framework for next-generation automated enterprise ingestion.

2. Problem Statement & Operational Gaps

2.1 The Core Problem: Ingestion Inaccuracy and Multi-Hop Misrouting

The downstream performance of an IT engineering or support team is bounded by the quality of upstream data injection. When a software developer or general staff member encounters an outage, bug, or access blockage, their primary objective is immediate documentation to resume execution. Consequently, the input is typically sparse (e.g., *"The staging DB is down," "Can't reach the authentication gateway,"* or *"My workspace tool is throwing a 502 error"*).

Standard enterprise ITSM implementations attempt to counter this by enforcing complex, multi-tiered forms requiring the user to self-select configuration items (CIs), target assignment groups, impact matrices, or architectural subcomponents. This strategy consistently fails; non-technical users lack the systemic visibility to correctly select infrastructure components, while technical users actively bypass complex drop-downs to save time. This mismatch results in generic classifications (e.g., mapping all software errors to "General IT Helpdesk"), which initiates an extensive, manual triage chain. A ticket may bounce between Network Ops, Database Administrations, Platform Security, and specific application squads before landing with the team that possesses the context and jurisdiction to resolve it.

2.2 Gaps in Existing Enterprise Frameworks

While proprietary ITSM platforms offer native AI add-ons, deep architectural evaluation reveals critical gaps that prevent broad market viability or strict corporate alignment:

Dimension	Current Native/Alternative Approaches	The Ticketris Architecture Gap Resolution
Data Privacy & Model Training	Requires the fine-tuning or continuous retraining of LLMs or traditional ML models directly on sensitive enterprise historical data tickets and internal logs.	Zero-Training Runtime RAG: Corporate data remains safely localized. Context is dynamically compiled at execution time via secure, ephemeral MCP queries.
Dynamic Tool Orchestration	Relies on static IntegrationHub spokes or custom webhooks that must be hardcoded for specific field mappings and exact API payloads.	Agentic MCP Discovery: The LLM reads standardized MCP tool schemas dynamically, autonomously deciding which system to query based on contextual evolution.
Vendor and Tool Lock-in	AI features are deeply bound to specific platform licensing models (e.g., ServiceNow Now Assist Pro/Enterprise) and cannot bridge disparate systems.	Platform-Agnostic Middleware: Standardizes input and output protocols using an open-source layer, allowing seamless cross-platform operational transfers.
Data Recency & Drift	Model weights or vector indices lag behind real-time corporate infrastructure shifts, leading to stale routing recommendations.	Transactional Live Polling: Queries live CMDB and version control metadata at the precise moment of intake, preventing architectural drift errors.

Critical Enterprise Risk Avoided: Model Poisoning and Exposure

Training internal LLMs on historical enterprise tickets often captures legacy configuration items, outdated team names, and hardcoded secrets or PII accidentally pasted into past incidents. By strictly avoiding training and adopting a protocol-driven live-lookup mechanism, Ticketris inherently guarantees compliance with modern data security standards (GDPR, SOC2, HIPAA).

3. High-Level System Architecture

The Ticketris framework is architected as an intermediary, decoupled state machine that coordinates communication between an abstract UI layer, an LLM orchestrator, a decentralized layer of Model Context Protocol servers, and the destination ITSM platform.

Figure 3.1: Structural Topology of the Ticketris Orchestration Framework

3.1 Core Architecture Components

- **Intake Interface (Abstraction Layer):** A thin UI engine deployed as a web panel, a Slack/Teams app, or an embedded widget inside an existing portal. It reads the raw user string and handles the rendering of the secondary, enriched review form returned by the core framework.
- **Ticketris Core Orchestrator:** A technology-neutral, stateful runtime system that wraps an external or internal open LLM execution context. It implements the Model Context Protocol Client capabilities, managing tool discovery, conversation state, call routing, context stitching, and formatting control.
- **MCP Gateway/Server Matrix:** A series of standalone micro-agents exposing standardized JSON-RPC endpoints over standard input/output (stdio) or secure transport layers (SSE). Each server isolates access to a specific database (CMDB, Identity Access Directories, Version Control Metadata, Knowledge Bases).
- **Staging & Human-in-the-Loop Engine:** A state machine that stores the generated rich payload in an ephemeral data store, withholding submission to the final ticket tracking system until a verification cryptographic token or UI confirmation event is dispatched by the user.
- **ITSM Connector Bridge:** A modular outbound gateway mapping the verified internal Ticketris structural JSON payload directly to the proprietary target APIs (e.g., ServiceNow Table API, Jira REST API v3).

4. Technology-Neutral Implementation Blueprint

4.1 Phase 1: Ingestion and Intent Parsing

The ingestion lifecycle initiates when the interface pushes a raw text payload containing an unstructured user statement along with the authenticated metadata of the reporting individual (e.g., corporate email, global unique employee ID, and location domain).

Upon receiving the input, the Core Orchestrator boots an ephemeral execution frame. It injects a highly controlled system instruction set that forces the underlying LLM to execute an initial parsing pass. The objective of this pass is not to solve the issue, but to extract primary entities and formulate initial hypothesis spaces regarding the nature of the technical fault. The framework explicitly forbids the LLM from attempting to generate field assumptions without tool-based cross-referencing.

4.2 Phase 2: The Model Context Protocol (MCP) Execution Loop

Rather than configuring custom programmatic integration chains for every possible enterprise database configuration, Ticketris standardizes internal querying via the Model Context Protocol. The Core Engine queries its locally connected MCP servers for their exposed tool schemas.

The conversation state loops programmatically through an agentic cycle:

1. The Orchestrator provides the raw user description and user metadata to the LLM alongside the available tool definitions fetched from the local MCP servers.
2. The LLM evaluates the payload and outputs an execution array of one or more tool calls formatted as standardized JSON-RPC objects.
3. The Orchestrator intercepts these calls, paths them to the corresponding local MCP server (e.g., querying the asset database for laptops assigned to the specific user unique ID), awaits the database execution payload, and pipes the structured results back into the LLM context block.
4. This loop continues until the LLM determines it has achieved maximal resolution context or hits a hard deterministic frame limit (configured natively at a maximum of three iterations to prevent execution runaways).

4.3 Phase 3: Structural Context Stitching and Disambiguation

A frequent complication occurs when a user's statement maps to ambiguous asset profiles within the enterprise. For instance, if the database returns three active application services matching the keyword "Payment Gateway" (e.g., `payment-gw-stg`, `payment-gw-prod-eu`, `payment-gw-prod-us`), the Core Orchestrator forces a strict multi-choice model generation.

Instead of predicting the target service with low structural certainty, the Ticketris framework instructs the model to populate an array of potential entities within a `disambiguation_required` block in the internal

JSON manifest. This array is translated by the UI layer into a selectable drop-down menu during the human review state, shifting the cognitive load of disambiguation safely to the user via an intuitive user interface framework.

4.4 Phase 4: Human-in-the-Loop (HITL) Staging & Verification

Once the runtime context enrichment loop terminates, Ticketris transforms the unstructured query into an enterprise-grade structural blueprint. The orchestrator structures a response mapping to standard incident objects: Category, Configuration Item, Target Assignment Group, Priority Matrix calculation, Detailed Description Summary, and Next Action Steps.

This object is stored in a transactional staging state cache. The interface renders these properties back to the end user with visual indicators denoting fields populated via the automated background lookup. The user retains complete administrative control to overwrite assignment groups, adjust urgency levels, or specify the exact asset. The ticket remains completely blocked from entering the production system queues until the explicit `COMMIT_TICKET` event payload is dispatched.

5. Technical Specifications & Schema Standards

To ensure compatibility with open-source systems and cross-enterprise deployments, Ticketris enforces clear JSON schemas for all communication blocks passing through the Model Context Protocol boundary.

5.1 Sample MCP Tool Definition Schema (Exposed to Orchestrator)

The following structure outlines how an internal enterprise CMDB micro-agent advertises its capabilities to the Ticketris Core Engine via the Model Context Protocol standard:

```

{
  "name": "query_user_assets",
  "description": "Queries the enterprise asset database to retrieve active devices, virtual machines, and cloud environments assigned to a specific user identifier.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "employee_id": {
        "type": "string",
        "description": "The globally unique employee identifier format (e.g., EMP-883102).",
      },
      "asset_type": {
        "type": "string",
        "enum": ["hardware", "virtual_machine", "saas_entitlement", "all"],
        "description": "Filter criteria for narrowing resource types."
      }
    },
    "required": ["employee_id"]
  }
}

```

5.2 Sample Intermediary Staging Payload Schema (Core to UI View)

This schema represents the highly structured payload generated by Ticketris after executing the background context extraction loops across the local MCP network. It contains both structured parameters and user-facing explanations:

```

{
  "ticketris_version": "1.0.0",
  "staging_id": "stg_tx_992104881_alpha",
  "source_user": {
    "employee_id": "EMP-4102",
    "email": "dev_lead@enterprise.com",
    "detected_location": "US-EAST-01"
  },
  "raw_ingested_input": "Internal authentication microservice is throwing token
verification failures when hit from the Austin test environment.",
  "enriched_fields": {
    "primary_category": "Infrastructure Engineering",
    "sub_category": "Identity & Access Management / OAuth2 Core",
    "predicted_assignment_group": "IAM-Core-Squad",
    "target_configuration_item": "auth-token-validator-svc-stg",
    "impact_level": "3-Medium",
    "urgency_level": "2-High",
    "calculated_priority": "3"
  },
  "context_lineage": [
    {
      "mcp_server": "mcp-server-iam",
      "tool_invoked": "resolve_service_ownership",
      "retrieved_records": 1,
      "confidence_score": 0.98
    },
    {
      "mcp_server": "mcp-server-cmdb",
      "tool_invoked": "query_environment_status",
      "retrieved_records": 3,
      "confidence_score": 0.92
    }
  ],
  "disambiguation_required": {
    "field": "affected_environment_cluster",
    "options": [
      {"id": "clus-austin-test-01", "label": "Austin In-House Testing Cluster (Active
Latency)"},
      {"id": "clus-austin-dev-backup", "label": "Austin Developer Secondary Failover
Cluster"}
    ],
    "requires_user_selection": true
  },
  "generated_documentation": {
    "synthesized_summary": "Automated Context Summary: The user reports systemic token
validation failures originating from the Austin physical testing facility. Real-time MCP
verification against the identity infrastructure stack confirms a configuration divergence
on the 'auth-token-validator-svc-stg' tracking branch. Cross-referencing active code
repositories maps ownership of this microservice straight to the IAM-Core-Squad.",
    "matched_kb_articles": [
      {

```



```
    "article_id": "KB-99120",
    "title": "Resolving OAuth2 Clock Drift in Decentralized Regional Edge Clusters",
    "relevance": "High"
  }
]
},
"system_state": "WAITING_FOR_USER_REVIEW"
}
```

6. Data Privacy, Governance & Compliance

6.1 Zero-Training Isolation Mechanics

A non-negotiable principle of the Ticketris framework is the total exclusion of enterprise production metrics, access tokens, code bases, or private internal infrastructure catalogs from any machine learning training routines. Traditional enterprise solutions leverage customer ticket databases to execute supervised classification training runs. This approach exposes the enterprise to severe threat vectors, including data inversion exploits, unintended inference of corporate intellectual property by external base-model APIs, and high long-term maintenance overhead to counter data drift.

Framework Operational Standard

Ticketris functions entirely within ephemeral transactional execution bounds. The LLM handles tasks as a semantic runtime compiler. It reads the schema definitions, executes tool calls locally, and maps structures based entirely on real-time contexts. Once the token validation sequence completes and the ticket is pushed to the target ITSM via outbound API hooks, the prompt context frames are explicitly wiped from memory.

6.2 Role-Based Access Control (RBAC) Propagation

Ticketris mitigates the risk of privilege escalation via prompt injection by ensuring that all underlying MCP micro-servers operate under a delegated security architecture. When Ticketris orchestrates an MCP tool execution loop on behalf of a user, the MCP gateway forwards the user's cryptographic OAuth Identity Token down to the specific data server.

If the end user does not possess authorization inside the underlying enterprise infrastructure platform to see a specific environment variable, database entry, or repository manifest, the corresponding MCP server returns a standard `403 Unauthorized` response code block within the protocol wrapper. The LLM reads this constraint naturally as an environmental limitation and informs the user or shifts to alternative open queries, ensuring the AI model never acts as an illegal back-door accessor to secured internal resources.

7. Deployment, Scaling & Runbook Operations

7.1 Network Topology and Protocol Lifecycles

Ticketris is designed for containerized orchestration deployments (e.g., Docker, Kubernetes blueprints). The core framework engine runs inside an isolated security zone with ingress paths restricted to authenticated frontend web panels or chat webhook controllers. The egress boundary splits into two distinct segments: public/private LLM inference endpoints via highly restricted TLS tunnels, and internal network connections leading to the private MCP server cluster.

7.2 Failover, Latency Management, and Edge Caching

To prevent the agentic validation loop from introducing unacceptable latency overhead during high-volume incident events, Ticketris implements a strict, multi-tiered timeout and caching matrix across the engine execution lifecycle:

Execution Stage	Deterministic Timeout Limit	Caching Strategy Enforced	Graceful Degradation / Fallback Behavior
MCP Schema Discovery	500 Milliseconds	Local Redis Cache (TTL: 24 Hours)	Load core framework schemas from a static, local local disk snapshot manifest.
Live Asset Query (CMDB)	2.5 Seconds	Ephemeral User Cache (TTL: 15 Minutes)	Bypass real-time lookup; present user with open text fields for manual entry.
Knowledge Base Search	3.0 Seconds	Semantic Vector Index Cache (TTL: 1 Hour)	Omit the 'Suggested Solutions' pane from final draft and proceed with routing triage.
LLM Output Synthesis	4.5 Seconds	None (Completely dynamic processing loop)	Abort agent execution frame, pull standard legacy routing matrix, display basic form.

8. Open Source License & Community Governance

8.1 GNU General Public License v3.0 Enforced Standard

The Ticketris framework, including its core state machine architecture, its open Model Context Protocol adaptation modules, and all included reference implementations of MCP micro-servers (CMDB, IAM,

Knowledge Base connectors), is globally licensed under the **GNU General Public License v3.0 (GPL-3.0)**.

This legal frameworks guarantees that any organization, enterprise, or independent developer can freely inspect, download, host, modify, and execute Ticketris within their corporate infrastructures. However, under the reciprocal share-alike parameters of the GPL-3.0 agreement, any derivative versions, customized core components, or extended distribution architectures of the Ticketris framework created by developers and shared publicly must be completely open-sourced under the exact same GPL-3.0 licensing terms. This ensures the structural ecosystem remains insulated from proprietary capture, fostering communal collaboration across enterprise infrastructure teams.

8.2 Framework Reference Architecture Legal Copy

GNU GENERAL PUBLIC LICENSE Version 3, 29 June 2007

Copyright (C) 2026 Ticketris Open Source Framework Authors <<https://github.com/ticketris/framework>>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble: The GNU General Public License is a free, copyleft license for software and other kinds of works. This technical specification framework and its implementation software are fully bound by its provisions. Protecting your operational freedom and data sovereignty is the primary goal of this decentralized, open-source integration pipeline standard.